

Project - High Level Design

on

Healthcare Content Generator

Course Name: Generative AI

Institution Name: Medicaps University – Datagami Skill Based Course

Student Name(s) & Enrolment Number(s):

Sr no	Student Name	Enrolment Number
01	MEET SOLANKI	EN22CS301595
02	MAYURI PATIDAR	EN22CS301593
03	MOKSHIKA YADAV	EN22CS301614
04	PALAK SHAHDADPURI	EN22CS301677
05	NIRDESH SHARMA	EN22CS301655
06	PANKAJ	EN22CS301679

Group Name: 11D6

Project Number: GAI-22

Industry Mentor Name:

University Mentor Name: Prof. Ajaj Khan

Academic Year: 2025-26

Table of Contents

1. Introduction.
 - 1.1. Scope of the document.
 - 1.2. Intended Audience
 - 1.3. System overview.
2. System Design.
 - 2.1. Application Design
 - 2.2. Process Flow.
 - 2.3. Information Flow.
 - 2.4. Components Design
 - 2.5. Key Design Considerations
 - 2.6. API Catalogue.
3. Data Design.
 - 3.1. Data Model
 - 3.2. Data Access Mechanism
 - 3.3. Data Retention Policies
 - 3.4. Data Migration
4. Interfaces
5. State and Session Management
6. Caching
7. Functional Requirements
 - 7.1. User Authentication & Authorization
 - 7.2. Infrastructure Configuration Management
 - 7.3. Provisioning Control & Monitoring
 - 7.4. Telecom Service Orchestration
 - 7.5. Dockerfile (Application Containerization)
 - 7.6. GitHub Actions (Workflow Configuration)
 - 7.7. CI/CD Strategy (Continuous Integration & Deployment)
8. Non-Functional Requirements
 - 8.1 Security Aspects
 - 8.2 Performance Aspects
9. References
10. GitHub
11. Snapshots

1. Introduction

1.1 Scope of the Document

This document provides a High-Level Design (HLD) for the Healthcare Content Generator system. It describes the system architecture, components, workflows, data handling, and integration with external services such as Groq Llama and ChromaDB. The document serves as a blueprint for developers, testers, and stakeholders to understand the system design and its functionalities.

1.2 Intended Audience

- Developers and Software Engineers working on the frontend using Streamlit and backend logic in Python
- System Architects designing the overall GenAI and RAG-based architecture using Groq Llama and ChromaDB
- QA/Test Engineers responsible for validating content generation, accuracy, and system workflows
- DevOps Engineers managing deployment, API integrations, and system performance
- Project Evaluators (Academic/Industry) reviewing the system design, implementation, and functionality.

1.3 System Overview

- The Healthcare Content Generator is a Python-based application with a Streamlit interface that allows users to input a medical topic and generate structured healthcare content. It uses Groq Llama along with a RAG pipeline powered by ChromaDB to produce outputs such as

patient summaries, clinical notes, and discharge instructions based on retrieved medical context.

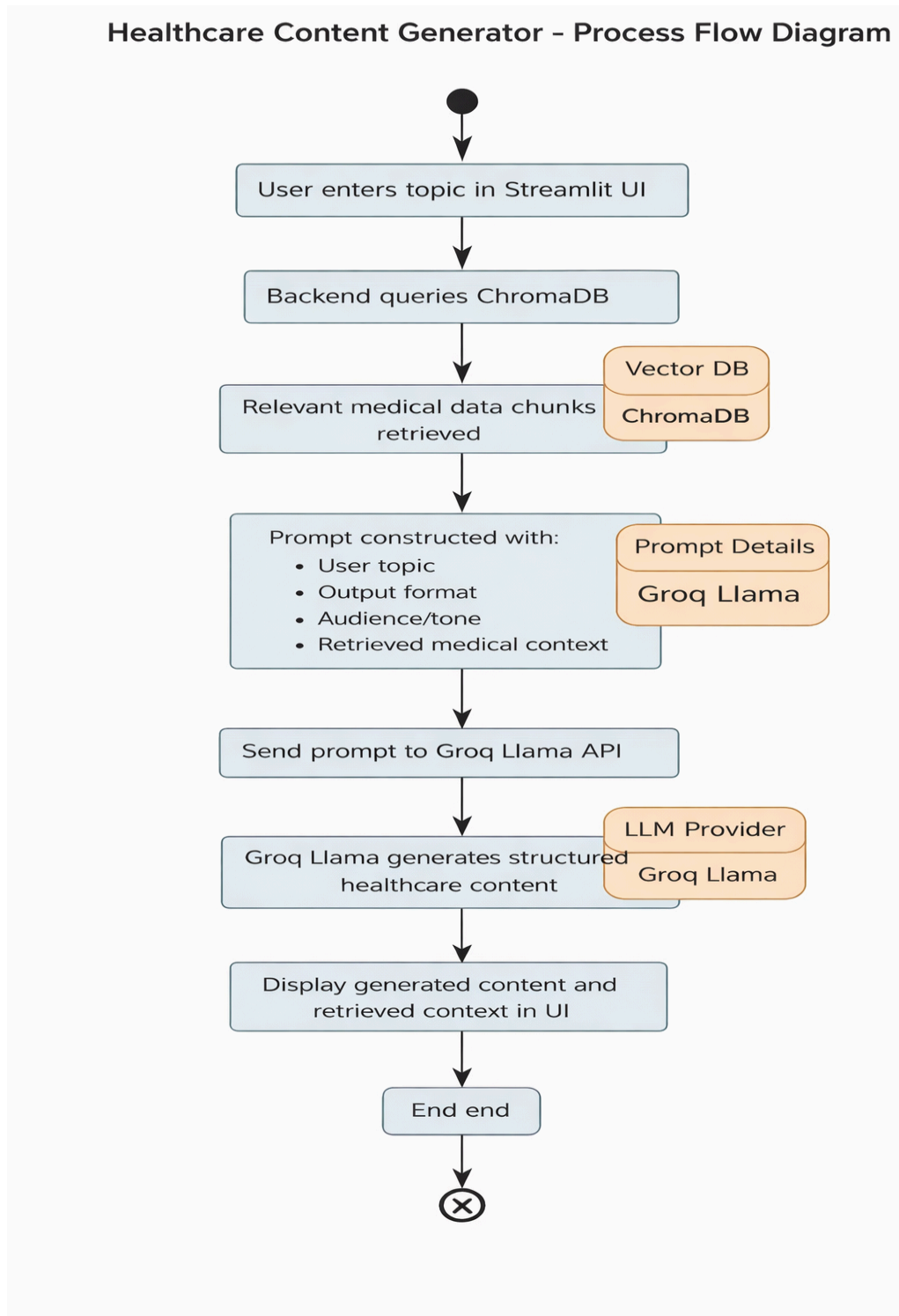
2. System Design

2.1 Application Design

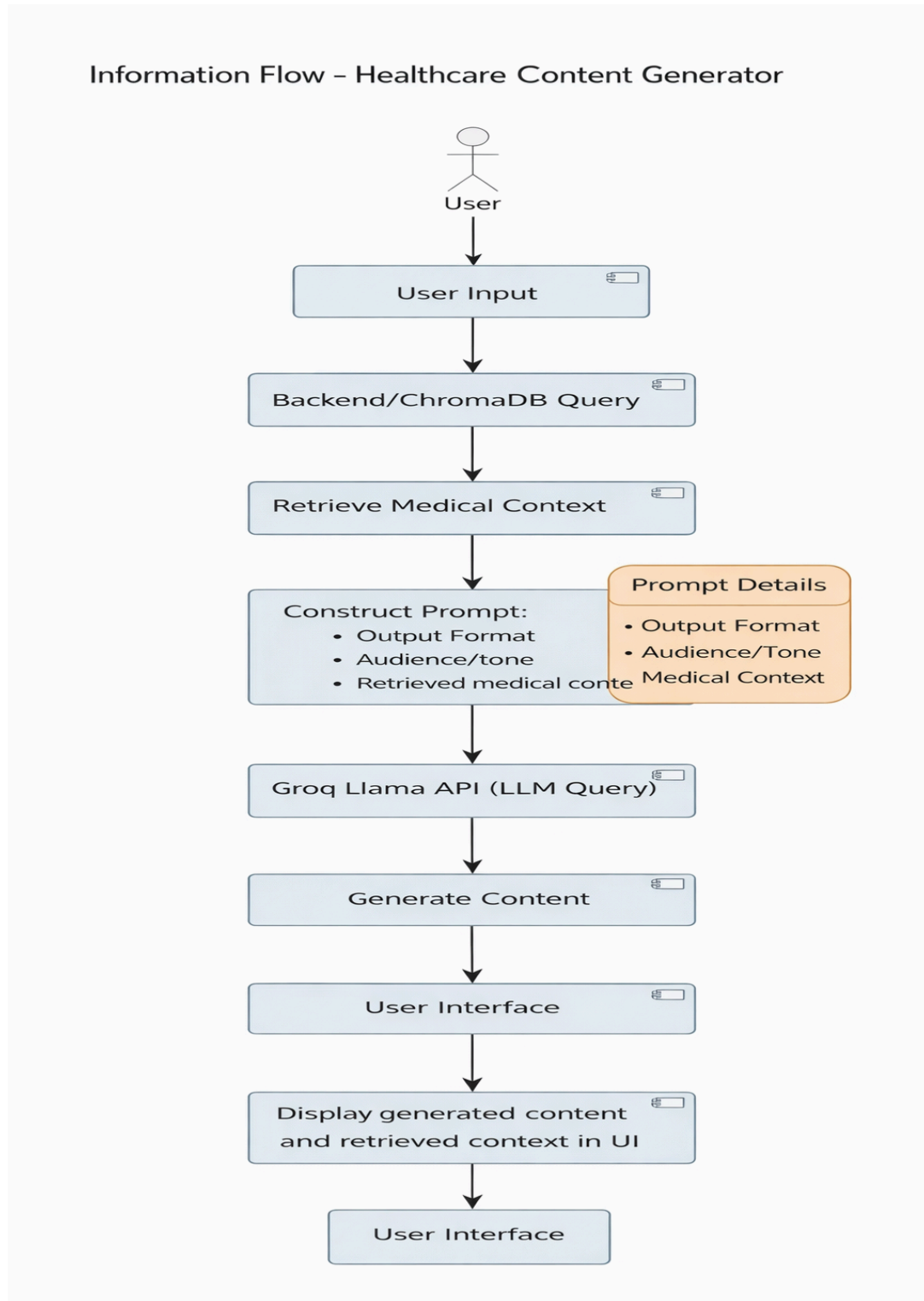
The application follows a layered architecture:

- **Presentation Layer:** Streamlit UI for user input and output display
- **Application Layer:** Python backend handling request flow and prompt construction
- **Service Layer:** LLM orchestration using Groq Llama
- **Data Layer:** Retrieval using ChromaDB for RAG
- **Configuration Layer:** Environment-based settings and API configurations

2.2 Process Flow



2.3 Information Flow



2.4 Components Design

Frontend (UI)

- Streamlit interface for user input and displaying generated content

Backend

- Python-based application handling request flow and prompt construction

Service Layer

- LLM Orchestrator for managing generation flow
- Integration with Groq Llama

Data Layer

- ChromaDB for storing and retrieving medical context (RAG)

Configuration

- Environment variables (.env)
- API key and settings management

External Systems

- Groq Llama API for content generation

2.5 Key Design Considerations

- **Stateless Architecture:** No persistent database; uses ChromaDB only for retrieval
- **Modular Design:** Clear separation between UI, backend logic, RAG pipeline, and LLM integration
- **LLM Integration:** Flexible use of Groq Llama for content generation
- **Error Handling & Validation:** Ensures valid user input and smooth workflow execution
- **Security of API Keys:** Managed via environment variables (.env)

- **Scalability:** Can be scaled and deployed using containerization techniques

2.6 API Catalogue

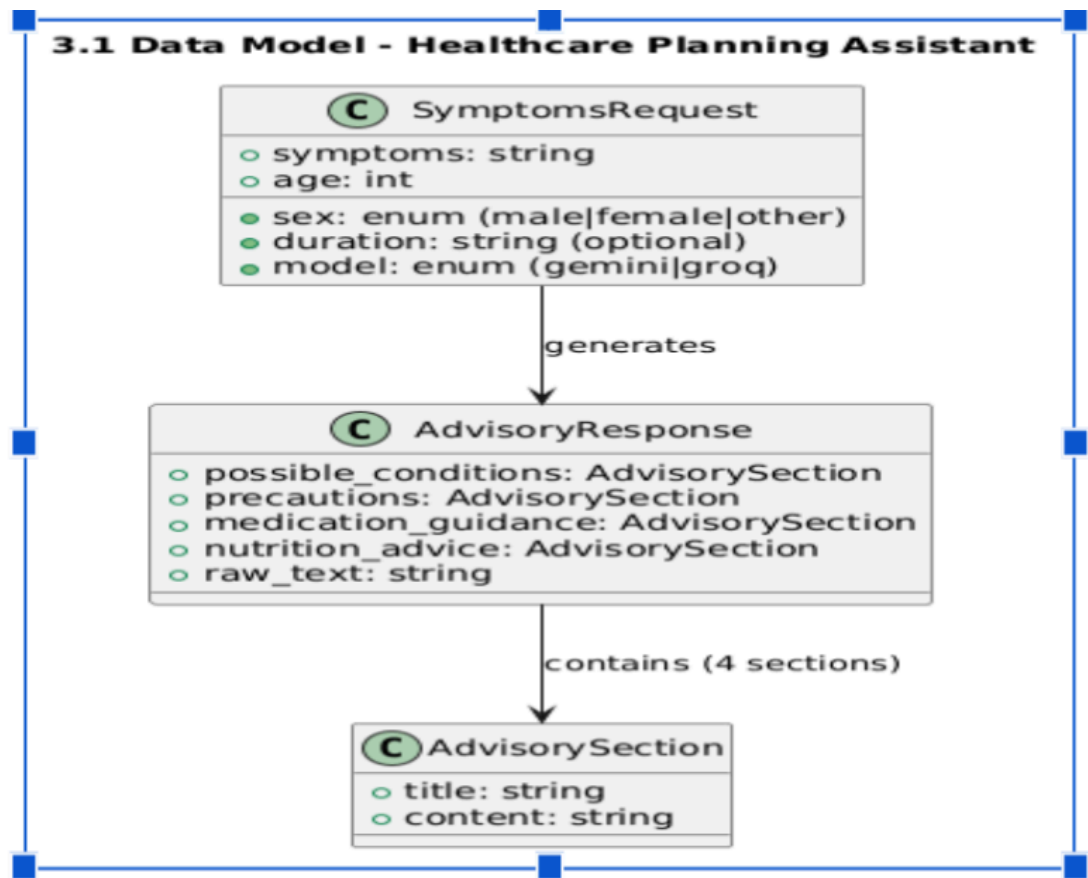
Endpoint	Method	Description
/	GET	Load Streamlit UI
/generate	POST	Generate healthcare content from user topic
/health	GET	Health check of the application

3. Data Design

3.1 Data Model

No persistent database is used. Data is handled via in-memory objects:

- ContentRequest (user topic, format, tone)
- ContentResponse (generated healthcare content + retrieved context)



3.2 Data Access Mechanism

- Data is passed via user input in Streamlit
- Processed in-memory within the Python backend
- Retrieval performed using ChromaDB (RAG pipeline)

3.3 Data Retention Policies

- No data is stored permanently
- API keys are managed via environment variables (.env)
- Generated responses exist only during the request lifecycle

4. Interfaces

- **User Interface:** Web-based UI built using Streamlit
- **API Interface:** Internal Python-based request handling (no external REST APIs)
- **External APIs:**
 - Groq Llama API for content generation
 - ChromaDB for retrieval of medical context

5. State and Session Management

- The system is stateless
- No session storage or cookies are used
- Each request is processed independently

6. Caching

Currently not implemented

Future Enhancement:

- Cache repeated queries and responses
- Use caching tools like Redis for performance optimization

7. Functional Requirements

7.1 User Authentication & Authorization

- Not implemented in the current version
- Future scope: login/signup system

7.2 Infrastructure Configuration Management

- Managed via environment variables (.env)
- Includes API keys and model settings

7.3 Provisioning Control & Monitoring

- Basic health check endpoint (/health)
- Logging for debugging and monitoring

7.4 Service Orchestration

- Handles prompt construction, retrieval, and LLM response flow
- Integrates Groq Llama with ChromaDB

7.5 Dockerfile (Application Containerization)

- Application can be containerized using Docker
- Ensures portability and consistent deployment

7.6 GitHub Actions (Workflow Configuration)

- Automates build, test, and deployment
- Supports CI workflows

7.7 CI/CD Strategy

- **Continuous Integration:** Code validation and testing
- **Continuous Deployment:** Automated deployment to server/cloud

8. Non-Functional Requirements

8.1 Security Aspects

- API keys are managed securely via environment variables and not exposed
- Input validation to prevent invalid or unsafe inputs
- Basic request handling safeguards in the backend
- Protection against malformed or incorrect user inputs

8.2 Performance Aspects

- Lightweight architecture with no persistent database reduces overhead
- Efficient LLM calls via Groq Llama
- Faster retrieval using ChromaDB (RAG pipeline)

9. References

9. References

- Streamlit Documentation
- ChromaDB Documentation
- Groq Llama API Documentation
- Group Name: **11D6-** Healthcare Content Generator

10. Github

- Github link -
<https://github.com/Nirdesharma/Healthcare-Content-Generator>

11. Snapshots

